

# Циклы

Повторять действия через `for` и `while`, понимать `range()`, счётчик, `break` и `continue`.

DAY 10

## ITERATION

for

while

range

break

ЦЕЛЬ ДНЯ

# Что ты сможешь после этого дня

Конкретные действия вместо списка терминов.

01 Повторять действия через `for` и `while`, понимать `range()`, счётчик, `break` и `continue`.

02 Объяснить минимум 5 терминов темы своими словами.

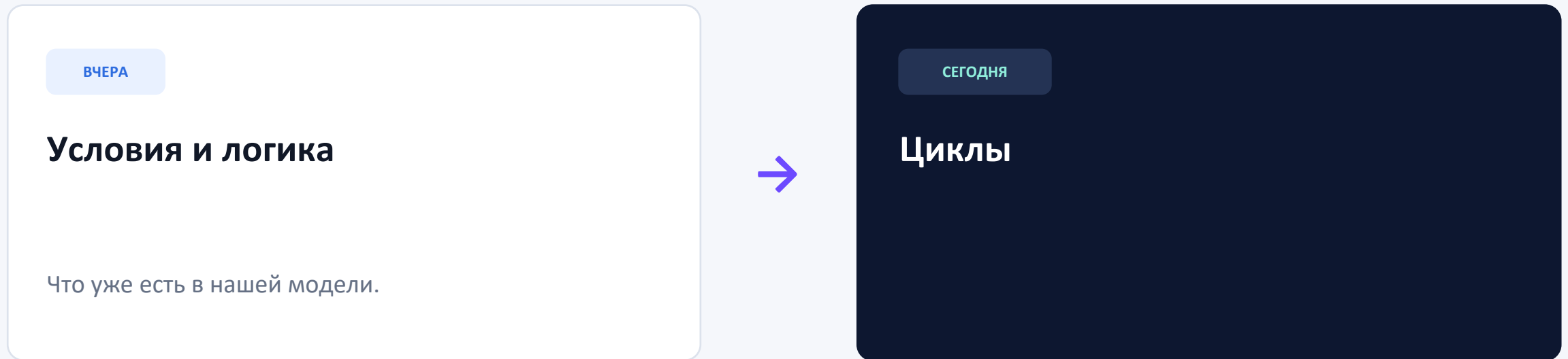
03 Прочитать показанный пример кода и предсказать результат.

04 Самостоятельно решить похожую задачу после разбора.

СВЯЗЬ

# Новая тема не появляется из ниоткуда

Связываем сегодняшний шаг с уже построенной картой курса.



Вчера: Условия и логика. Сегодня мы добавляем следующий слой: циклы.

ЗАЧЕМ

# Зачем мне это?

Каждая тема решает конкретную проблему.

ПРОБЛЕМА → ПОЛЬЗА

**Циклы убирают копирование одинакового кода и позволяют обрабатывать наборы данных любого разумного размера.**

Вопрос себе

**Какую реальную проблему решает сегодняшняя тема?**

Ответь до следующего слайда.

# Сначала — человеческий образ

Потом обязательно связываем его с настоящей техникой.

## БЫТОВОЙ ОБРАЗ

**Цикл — конвейер: одно и то же действие проходит по нескольким элементам. `while` похож на «повторяй, пока условие выполняется».**

## ПЕРЕВОД В IT

**iteration**

одно выполнение тела цикла

**for**

цикл перебора элементов последовательности

**while**

цикл, работающий, пока условие истинно

# Слой 1: iteration • for • while

Сначала смысл, затем профессиональный термин.

**iteration**

одно выполнение тела цикла

**for**

цикл перебора элементов последовательности

**while**

цикл, работающий, пока условие истинно

## Слой 2: range • break • continue

Сначала смысл, затем профессиональный термин.

**range**

объект, задающий последовательность целых чисел

**break**

оператор немедленного выхода из цикла

**continue**

оператор перехода к следующей итерации

## Слой 3: counter • infinite loop

Сначала смысл, затем профессиональный термин.

**counter**

переменная для подсчёта повторений

**infinite loop**

цикл, условие завершения которого никогда не наступает

Не заучивай формулировку. Придумай свой пример использования каждого термина.

## Слой 4: nested loop • enumerate

Сначала смысл, затем профессиональный термин.

**nested loop**

цикл, расположенный внутри другого цикла

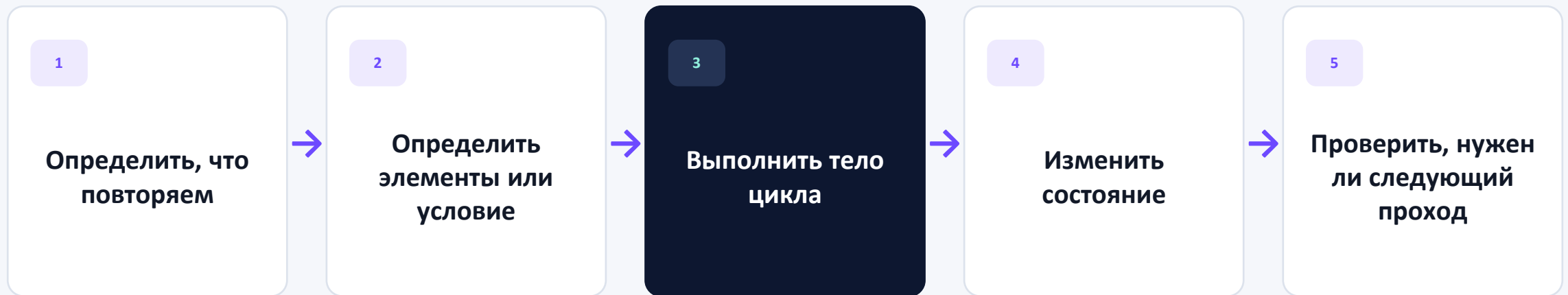
**enumerate**

получение индекса и значения при обходе последовательности

Не заучивай формулировку. Придумай свой пример использования каждого термина.

# Как это работает по шагам

Проследи движение слева направо.



Проговори эту цепочку своими словами, не подсматривая в текст.

# Первый минимальный пример

Смотрим только на одну основную идею.



day10\_step1

```
total = 0
for number in range(1, 6):
    total = total + number
print(total)
```

## ЧТО ИСКАТЬ

- Где данные?
- Где действие?
- Где результат?
- Что изменится, если заменить одно значение?

# Разбираем пример построчно

Код не должен оставаться магической конструкцией.

```
total = 0
```

Сохраняем значение или результат вычисления под понятным именем.

```
for number in range(1, 6):
```

Начинаем цикл и последовательно перебираем элементы.

```
total = total + number
```

Сохраняем значение или результат вычисления под понятным именем.

```
print(total)
```

Показываем результат, чтобы увидеть состояние программы.

РЕЗУЛЬТАТ

# Код → выполнение → результат

Всегда связываем запись с тем, что реально получилось.



до запуска

```
total = 0
for number in range(1, 6):
    total = total + number
print(total)
```

РЕЗУЛЬТАТ

15

ИЗМЕНИЛИ

# Теперь изменим пример

Сначала предскажи результат. Не запускай код сразу.



changed

```
count = 3
while count > 0:
    print(count)
    count -= 1
print("Старт!")
```

**Что появится в  
результате?**

Сформулируй ответ словами.

ОТВЕТ

# Сверяем прогноз

Не просто смотри ответ — объясни, почему он именно такой.

РЕЗУЛЬТАТ

3  
2  
1  
Старт!

ПОЧЕМУ

**Python/система выполняет шаги в записанном порядке. Изменённые данные или действие дают новый результат.**

Сравни с первым примером: что изменилось?

## ОШИБКА

# Как писать неправильно

Ошибка — отдельный учебный материал, а не повод её скрывать.



wrong

```
count = 3
while count > 0:
    print(count)
```

СТОП

**Что здесь может сломаться  
или ввести нас в  
заблуждение?**

Сначала найди проблему сам.

# Почему возникает проблема

Переводим техническую причину на обычный язык.

**count** никогда не меняется, поэтому условие всегда True. Получается бесконечный цикл.

Алгоритм отладки: симптом → конкретная строка/шаг → причина → минимальное исправление → повторная проверка.

# Исправленный вариант

Меняем только то, что действительно было причиной.



fixed

```
count = 3
while count > 0:
    print(count)
    count -= 1
```

## ПРАВИЛО

**Не переписывай всё решение. Найди минимальное осмысленное исправление.**

# Что изменилось после исправления?

Ответь без запуска.

- 1 Исчезла ли исходная причина проблемы?
- 2 Какой результат/поведение теперь ожидаем?
- 3 Как проверить это одним маленьким примером?

# Сначала увидим полностью разобранный пример

Только после него будет самостоятельная практика.

ЗАДАЧА

**Посчитать сумму только чётных чисел от 1 до 10.**

Не начинаем с кода. Сначала переводим задачу на человеческий язык.

КАК ДУМАТЬ

# Как рассуждает программист

Разбиваем большую формулировку на маленькие действия.

- 1 Перебираем числа 1..10.
- 2 Для каждого проверяем остаток от деления на 2.
- 3 Чётные добавляем в total.
- 4 Нечётные пропускаем.
- 5 После цикла печатаем сумму.

# Шаг 1: собираем основу

Не показываем 20 строк сразу — строим решение частями.



solution\_part1

```
total = 0
for number in range(1, 11):
    if number % 2 == 0:
```

## Проверь

- Какие данные уже есть?
- Какой шаг плана реализован?
- Что ещё осталось добавить?

ИТОГОВЫЙ КОД

## Шаг 2: итоговое решение

Теперь видим программу/запрос целиком.



```
total = 0
for number in range(1, 11):
    if number % 2 == 0:
        total += number
print(total)
```

solution\_final

# Проверяем результат

Сравниваем то, что ожидали, с тем, что получили.

РЕЗУЛЬТАТ

30

ГЛАВНОЕ

**Задача была решена не потому, что мы «знали готовый код», а потому что сначала построили понятный план.**

ПОПРОБУЙ САМ

# Практика 1

Задача очень похожа на разобрannую — но теперь первый ход за тобой.

**Сделай счётчик трёх попыток входа через `while`.**

Сначала выпиши 2–5 шагов решения словами.

ПОДСКАЗКА

## Подсказка — только направление

Попробуй ещё раз до просмотра готового решения.

**Начни `attempts = 3` и обязательно уменьшай значение внутри цикла.**

Вопрос себе: какие конструкции сегодняшнего дня нужны, а какие пока лишние?

# Один из правильных вариантов

Сравни логику, а не количество строк.



practice\_solution

```
attempts = 3
while attempts > 0:
    print("Осталось попыток:", attempts)
    attempts -= 1
print("Попытки закончились")
```

# Ожидаемый результат

Если твой код устроен иначе, но даёт правильное и понятное решение — это нормально.

## РЕЗУЛЬТАТ

Осталось попыток: 3  
Осталось попыток: 2  
Осталось попыток: 1  
Попытки закончились

## Сравни

- Результат
- Понятность имён
- Лишние шаги
- Обработку ошибок

## Практика 2 — мини-проект

Соединяем несколько элементов дня в одну маленькую закончённую задачу.

### ЗАДАНИЕ

**Перебери числа 1..10, пропусти 5 через `continue` и остановись на 8 через `break`.**

Попробуй сначала без подсказки. Разрешено подсматривать в сегодняшние примеры.

# Разбор мини-проекта

Вот один рабочий путь.

A dark-themed terminal window with a title bar at the top containing three colored circles (red, yellow, green) on the left and the text 'mini\_project' on the right. The terminal contains a Python code snippet.

```
for n in range(1, 11):  
    if n == 5:  
        continue  
    if n == 8:  
        break  
    print(n)
```

# Что должно получиться

Связываем итог с задачей и проверяем смысл.

РЕЗУЛЬТАТ

1  
2  
3  
4  
6  
7

Объясни каждую важную строку/команду своими словами. Если не можешь — вернись к конкретному месту, а не перечитывай весь день.

# Четыре ловушки новичка

Лучше увидеть их здесь, чем случайно закрепить как привычку.

01

**Забывать изменять условие `while`.**

02

**Ошибаться в границах `range()`.**

03

**Использовать `break` там, где нужен обычный `if`.**

04

**Не понимать, что `continue` пропускает остаток текущей итерации.**

# Проверь себя: 1–2

Не листай дальше, пока не сформулируешь ответ.

1

Что такое «iteration» и какую проблему это понятие помогает описать?

2

Чем «iteration» отличается от «range»? Объясни без копирования определения.

# Проверь себя: 3–4

Здесь проверяем понимание кода и процесса мышления.

- 3 Какой результат даёт первый минимальный пример и почему?
- 4 Какую ошибку/риск мы разобрали сегодня и какое минимальное исправление нужно?

## ОТВЕТЫ

# Ответы к вопросам 1–2

Формулировка может отличаться — важен смысл.

**1**    **iteration — одно выполнение тела цикла.**

**2**    **iteration: одно выполнение тела цикла. range: объект, задающий последовательность целых чисел.**

## ОТВЕТЫ

# Ответы к вопросам 3–4

Сверяем причинно-следственную связь.

**3**     **15**

**4**     **count** никогда не меняется, поэтому условие всегда True. Получается бесконечный цикл.

ИТОГ

# Сегодня мы научились

Не перечисляем слова — фиксируем новые способности.

- ✓ Повторять действия через `for` и `while`, понимать `range()`, счётчик, `break` и `continue`.
- ✓ Читать и объяснять пример темы «Циклы».
- ✓ Находить типичную ошибку и формулировать причину.
- ✓ Решать похожую задачу по шагам, а не угадывать готовый код.

[ДАЛЬШЕ](#)

# Что будет завтра

Связываем сегодняшний результат со следующим шагом.

## Списки и кортежи

Завтра: Списки и кортежи. Сегодняшняя модель станет опорой для следующего шага.

Сегодняшняя тема не заканчивается здесь — она станет инструментом в следующем дне.